LAPORAN PRAKTIKUM
JOBSHEET 9 - OPERASI LENGKAP CRUD PADA POSTGRESQL



Disusun Oleh:

Nama	:	Mohammad Hafidz Rafi' Rabbani
NIM	:	254107020084
Kelas	:	2H
No. Absen	:	14
Repository	:	github.com/hafidzrafi/DnPWeb2026

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan 3

2. Praktikum 3

2.1. Percobaan 1: Perancangan Antarmuka Tabel CRUD dan Aksi Baris 3

2.1.1. 1.1 Struktur Kolom Aksi pada Tabel 3

2.1.2. Langkah-langkah 4

2.1.3. Pertanyaan 5

2.2. Percobaan 2: Pengambilan Data Eksisting dan Form Pra-Terisi (Edit) 6

2.2.1. 2.1 Pengambilan Rekaman dan Pengisian Formulir 6

2.2.2. Langkah-langkah 7

2.2.3. Pertanyaan 8

2.3. Percobaan 3: Eksekusi Update Data dan Penghapusan Destruktif 9

2.3.1. 3.1 Pembaruan Data dengan UPDATE Statement 9

2.3.2. 3.2 Penghapusan Data Aman Berbasis HTTP POST 9

2.3.3. Langkah-langkah 10

2.3.4. Pertanyaan 11

3. Tugas 12

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 9 12

1. Tujuan

- Memahami siklus hidup lengkap arsitektur CRUD (Create, Read, Update, Delete) pada aplikasi web berbasis PostgreSQL.
- Menguasai alur pembacaan data eksisting melalui parameter URL HTTP GET untuk mengisi form pra-terisi pada edit.php.
- Mengimplementasikan pembaruan rekaman data (UPDATE) menggunakan Prepared Statements berparameter pada proses_edit.php.
- Menerapkan penggunaan input tersembunyi (<input type="hidden" name="id">) untuk pengikatan kunci utama entitas.
- Membangun penanganan operasi penghapusan data (DELETE) berbasis metode aman HTTP POST untuk mencegah eksekusi tak sengaja oleh crawler.
- Menerapkan penanganan kasus batas (edge cases) seperti parameter ID kosong atau ID tidak terdaftar di database.
- Mengintegrasikan umpan balik status transaksi (flash messages) pada setiap operasi mutasi data persisten.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: https://github.com/hafidzrafi/DnPWeb2026.

2. Praktikum

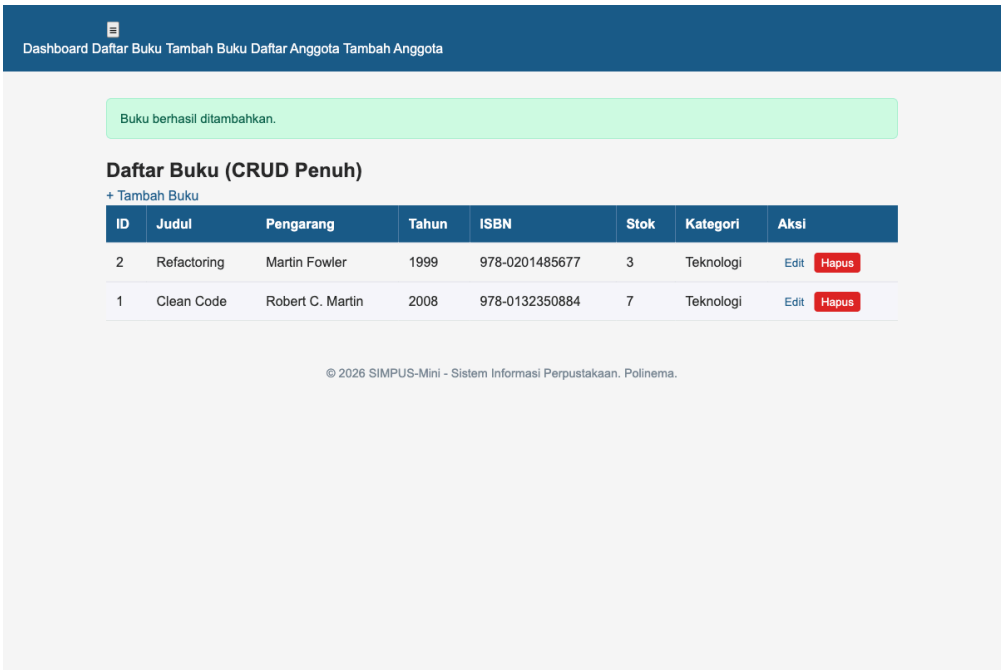
2.1. Percobaan 1: Perancangan Antarmuka Tabel CRUD dan Aksi Baris

Pada percobaan pertama, tabel daftar buku dan anggota pada Jobsheet 9 disempurnakan dengan menambahkan kolom Aksi yang memuat tombol interaktif Edit dan tombol Hapus berbasis formulir POST.

2.1.1. 1.1 Struktur Kolom Aksi pada Tabel

Setiap baris data dilengkapi dengan tautan navigasi menuju halaman formulir edit serta form mini untuk aksi destruktif:

```
<a href="edit.php?id=<?= urlencode($buku['id']) ?>" class="btn btn-secondary">Edit</a>
<form action="hapus.php" method="POST" style="display:inline;" onsubmit="return
confirm('Yakin ingin menghapus buku ini?');">
    <input type="hidden" name="id" value="<?= htmlspecialchars($buku['id']) ?>">
    <button type="submit" class="btn btn-danger">Hapus</button>
</form>
```



Gambar 1.1: Tampilan Tabel Daftar Buku Menampilkan Kolom Aksi dengan Tombol Edit dan Hapus Terpadu.

2.1.2. Langkah-langkah

1. Memodifikasi berkas `books/list.php` dan `members/list.php` dengan menambahkan kolom header `<th>Aksi</th>`.
2. Menyematkan tautan `edit.php?id=...` yang menyertakan identifier primary key pada setiap baris.
3. Menyematkan form mini dengan method POST untuk pemanggilan `hapus.php`.
4. Menguji kemunculan tombol aksi pada peramban web.

2.1.3. Pertanyaan

1. Mengapa identifier primary key pada URL wajib dienkode menggunakan `urlencode()`?

    - `urlencode()` memastikan karakter khusus atau spasi pada parameter URL diubah menjadi format representasi persen (**percent-encoded**) yang valid sesuai standar RFC 3986, mencegah terputusnya query string atau salah tafsir parameter oleh web server.

2.2. Percobaan 2: Pengambilan Data Eksisting dan Form Pra-Terisi (Edit)

Percobaan kedua mengimplementasikan alur pembacaan data sebelum proses modifikasi dilakukan pada berkas `edit.php`.

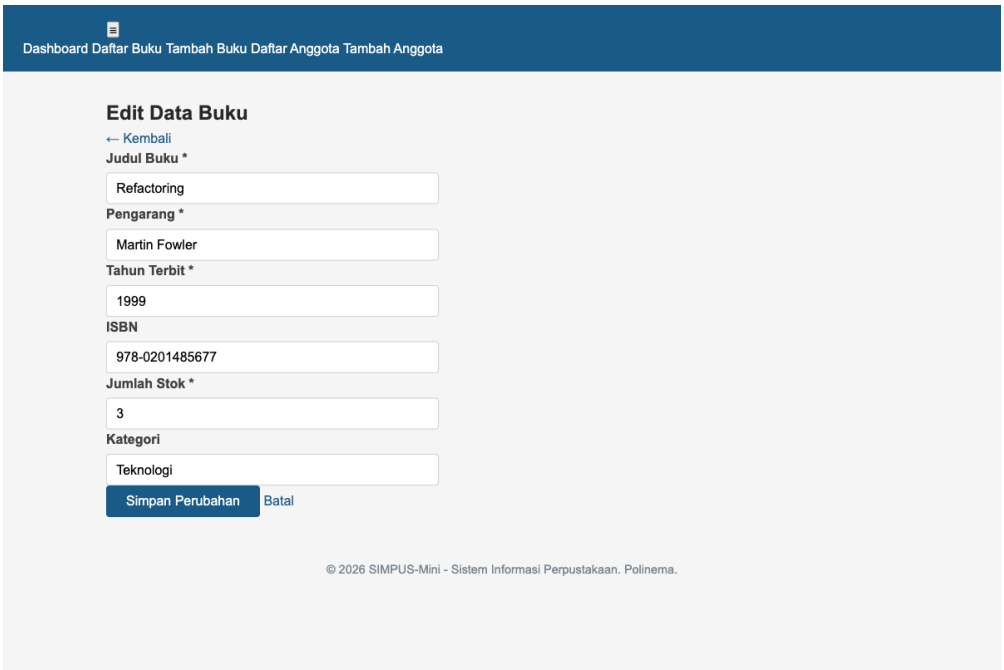
2.2.1. 2.1 Pengambilan Rekaman dan Pengisian Formulir

Skrip mengekstrak `$_GET['id']`, memvalidasi keberadaannya, lalu menjalankan query penelusuran `SELECT * FROM buku WHERE id = :id`. Data yang diperoleh kemudian disuntikkan ke dalam atribut `value` elemen input:

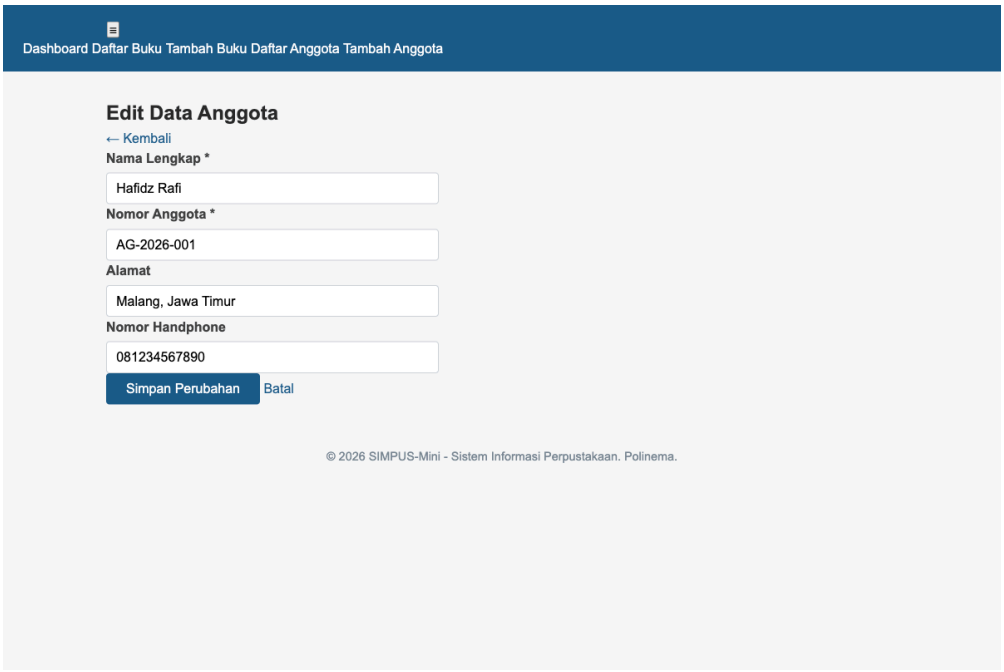
```
$id = $_GET['id'] ?? null;
if (!$id) {
    header('Location: list.php');
    exit;
}

$stmt = $pdo->prepare("SELECT * FROM buku WHERE id = :id");
$stmt->execute([':id' => $id]);
$buku = $stmt->fetch(PDO::FETCH_ASSOC);

if (!$buku) {
    header('Location: list.php');
    exit;
}
```



Gambar 1.2: Formulir Edit Buku dengan Nilai-Nilai Isian yang Terisi Otomatis Sesuai Data di Database.



Gambar 1.3: Formulir Edit Anggota yang Menampilkan Data Eksisting Anggota Terpilih.

2.2.2. Langkah-langkah

1. Membuat berkas formulir `books/edit.php` dan `members/edit.php`.
2. Mengekstrak parameter GET dan menjalankan query pencarian rekaman tunggal.
3. Menyematkan input tersembunyi `<input type="hidden" name="id" value="...">`.
4. Menguji pengisian form otomatis dengan mengklik tombol Edit pada tabel.

2.2.3. Pertanyaan

1. Apa fungsi elemen input tersembunyi `<input type="hidden" name="id">` pada formulir edit?

    - Elemen `<input type="hidden">` bertugas membawa nilai kunci utama (`id`) entitas yang sedang disunting saat formulir disubmit ke server via POST, tanpa menampilkannya sebagai kolom isian yang dapat diubah oleh pengguna pada antarmuka visual.

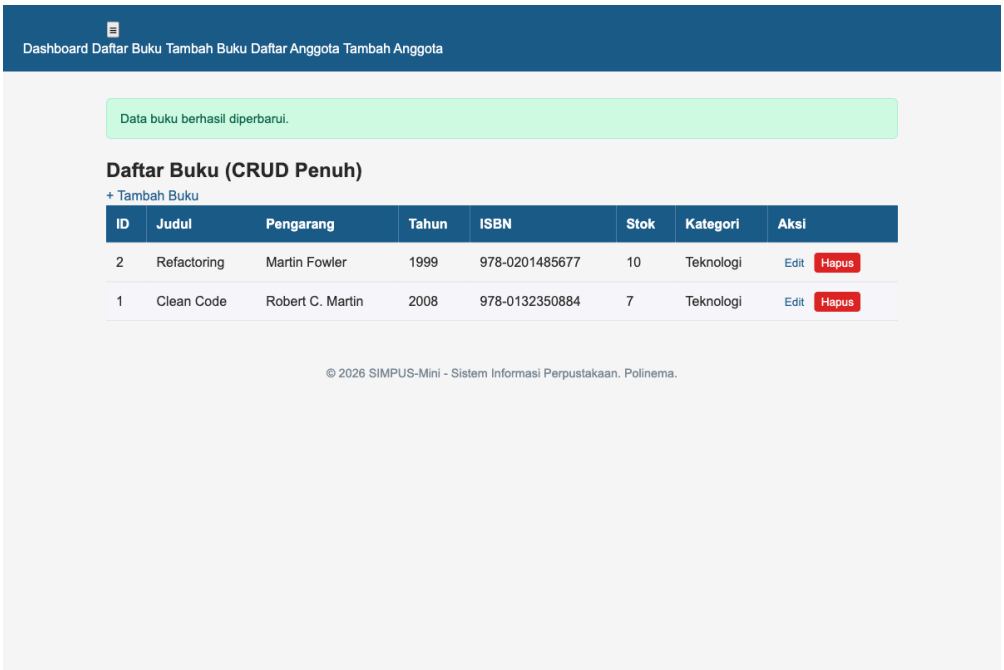
2.3. Percobaan 3: Eksekusi Update Data dan Penghapusan Destruktif

Percobaan ketiga mengimplementasikan pemrosesan pembaruan data (`UPDATE`) pada `proses_edit.php` serta eksekusi penghapusan data (`DELETE`) pada `hapus.php`.

2.3.1. 3.1 Pembaruan Data dengan UPDATE Statement

Skrip `proses_edit.php` memvalidasi input, lalu mengeksekusi perintah SQL pembaruan berparameter:

```
$stmt = $pdo->prepare(
    "UPDATE buku SET judul = :judul, pengarang = :pengarang, tahun = :tahun,
     isbn = :isbn, stok = :stok, kategori = :kategori WHERE id = :id"
);
$stmt->execute([
    ':judul' => $judul,
    ':pengarang' => $pengarang,
    ':tahun' => (int) $tahun,
    ':isbn' => $isbn,
    ':stok' => (int) $stok,
    ':kategori' => $kategori,
    ':id' => $id,
]);
```



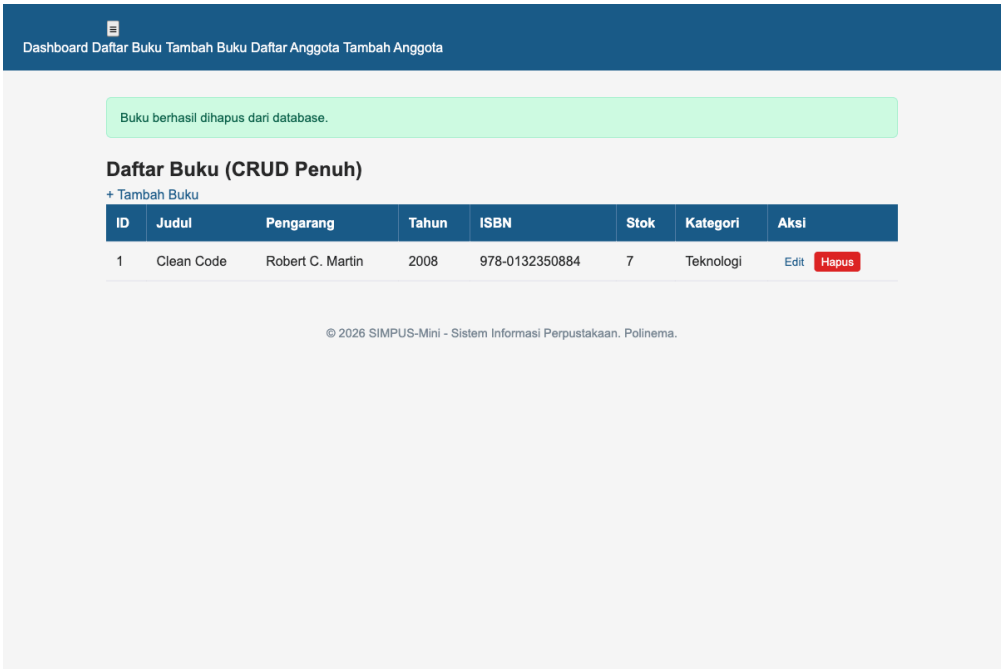
Gambar 1.4: Notifikasi Sukses Pembaruan Data Muncul dan Nilai Stok Buku Berhasil Terbarukan Menjadi 10.

2.3.2. 3.2 Penghapusan Data Aman Berbasis HTTP POST

Pada `hapus.php`, server memverifikasi bahwa metode request adalah POST untuk mencegah eksekusi tidak sengaja oleh bot perayap web (**web crawler**):

```php
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header('Location: list.php');
    exit;
}

$id = $_POST['id'] ?? null;
if ($id) {
    $stmt = $pdo->prepare("DELETE FROM buku WHERE id = :id");
    $stmt->execute([':id' => $id]);
}
```



Gambar 1.5: Baris Rekaman Buku Berhasil Dihapus Permanen dari Database Pasca-Konfirmasi Pengguna.

2.3.3. Langkah-langkah

1. Mengembangkan skrip `proses_edit.php` dengan query UPDATE berparameter.
2. Mengembangkan skrip `hapus.php` dengan pembatasan metode HTTP POST dan query DELETE.
3. Menguji perubahan nilai data dan memverifikasi pembaruan nilai di database dan antarmuka.
4. Menguji penghapusan baris data dan memverifikasi lenyapnya rekaman dari basis data.

2.3.4. Pertanyaan

1. Mengapa operasi penghapusan data dilarang menggunakan metode HTTP GET?
    - Metode GET bersifat aman dan idempoten menurut standar RFC. Penggunaan GET pada aksi destruktif membuka celah di mana perayap web (**web crawlers/search bots**) atau fitur **link prefetching** peramban dapat memicu penghapusan database secara tidak sengaja saat menelusuri tautan halaman.

3. Tugas

3.1. Tugas Evaluasi & Analisis Konseptual Jobsheet 9

Berikut adalah evaluasi kritis dan analisis mendalam terhadap lima pertanyaan evaluasi konsep pada Jobsheet 9:

No	Pertanyaan dan Jawaban Konseptual
1	**Soal:** Jelaskan siklus hidup lengkap operasi CRUD (Create, Read, Update, Delete) dalam arsitektur aplikasi web berbasis basis data relasional! Bagaimana pemisahan peran antara file formulir dan file pemroses mendukung arsitektur yang bersih? **Jawaban:** Siklus hidup CRUD mengintegrasikan empat manipulasi entitas inti: 1. **Create**: Pengguna mengirim data via formulir POST `tambah.php` -> diproses oleh `proses_tambah.php` menggunakan query `INSERT INTO`. 2. **Read**: Halaman `list.php` mengeksekusi `SELECT * FROM ...` dan menyajikan baris ke tabel HTML. 3. **Update**: Pengguna membuka `edit.php?id=...` yang membaca data eksisting -> formulir tersaji -> perubahan dikirim via POST ke `proses_edit.php` yang menjalankan `UPDATE ... WHERE id = :id`. 4. **Delete**: Pengguna memicu aksi form POST pada `hapus.php` yang menjalankan `DELETE FROM ... WHERE id = :id`. Pemisahan antara berkas antarmuka presentasi (`edit.php`, `list.php`) dan berkas pengendali mutasi (`proses_edit.php`, `hapus.php`) menegakkan prinsip **Separation of Concerns** (SoC), menghasilkan kode yang terisolasi, mudah dipelihara, dan terlindung dari kerentanan **form resubmission**.
2	**Soal:** Bagaimana alur kerja operasi Update data berjalan pada `edit.php` dan `proses_edit.php`? Mengapa input tersembunyi `<input type="hidden" name="id">` wajib disertakan di dalam formulir? **Jawaban:**

No	Pertanyaan dan Jawaban Konseptual
	Alur operasi Update dimulai ketika pengguna mengakses `edit.php?id=X`. Server membaca parameter GET, menarik data dari PostgreSQL melalui `SELECT ... WHERE id = :id`, dan menyuntikkan nilai atribut ke formulir. Saat formulir disubmit ke `proses_edit.php`, server membutuhkan penanda eksplisit mengenai baris mana yang harus diperbarui. Input tersembunyi `<input type="hidden" name="id" value="...">` bertugas membawa nilai primary key tersebut di dalam payload `$_POST` tanpa menampilkannya sebagai elemen yang dapat disunting oleh pengguna, sehingga query `UPDATE` dapat mengarahkan klausa `WHERE id = :id` secara presisi.
3	**Soal:** Mengapa operasi penghapusan data (`hapus.php`) sengaja dirancang hanya menerima metode HTTP POST dan dilarang keras menggunakan metode HTTP GET? **Jawaban:** Berdasarkan spesifikasi arsitektur HTTP, metode GET didefinisikan sebagai metode yang aman (**safe**) dan tidak boleh menimbulkan efek samping mutasi pada status server (**side-effects**). Jika penghapusan data menggunakan tautan GET (`hapus.php?id=1`), sistem rentan terhadap malapetaka data: robot perayap mesin pencari (seperti Googlebot), ekstensi peramban, atau fitur **link prefetching** otomatis pada peramban akan memicu eksekusi penghapusan rekaman secara masif tanpa disengaja. Menggunakan metode POST memastikan operasi penghapusan memerlukan itikad transaksi formulir yang eksplisit dari pengguna.
4	**Soal:** Bagaimana aplikasi menangani kasus batas (edge case) saat pengguna mencoba mengakses `edit.php` tanpa parameter `id` atau dengan `id` yang tidak terdaftar di database? **Jawaban:** Aplikasi menerapkan penanganan berjenjang melalui **guard clauses**:

No	Pertanyaan dan Jawaban Konseptual
	1. Pengecekan awal memeriksa `if (!isset($_GET['id']) \|\| empty($_GET['id']))`. Jika parameter absen, eksekusi langsung dialihkan ke `list.php` dengan `header('Location: list.php'); exit;`. 2. Jika parameter ada, query `SELECT * FROM buku WHERE id = :id` dijalankan. Jika ID tidak eksis di tabel, pemanggilan method `fetch()` akan menghasilkan nilai boolean `false`. 3. Kondisi `if (!$buku)` menangkap nilai false tersebut dan segera membelokkan alur kembali ke `list.php`, mencegah timbulnya peringatan PHP atau error indeks tidak terdefinisi (**Undefined array key**).
5	**Soal:** Jelaskan peran klausa `WHERE id = :id` pada perintah `UPDATE` dan `DELETE`! Apa bahaya katastropik yang terjadi jika klausa `WHERE` tidak sengaja terlewatkan? **Jawaban:** Klausa `WHERE id = :id` berfungsi sebagai filter pembatas cakupan mutasi agar perintah hanya diaplikasikan secara spesifik pada satu baris data yang ditargetkan. Jika klausa `WHERE` tidak sengaja terlewatkan: - Pada perintah `UPDATE buku SET stok = 0;`: Seluruh baris data di tabel buku tanpa terkecuali akan dimutasi serentak nilainya menjadi 0. - Pada perintah `DELETE FROM buku;`: Seluruh rekaman data di tabel akan dimusnahkan secara permanen (**accidental table wipeout**). Kehilangan klausa `WHERE` merupakan malapetaka integritas basis data yang berakibat fatal pada kerusakan data operasional.